PROJECT REPORT

TASK 18: SELECT DATASET COLUMNS USING PANDAS

1. Abstract

This project demonstrates how to select specific rows and columns from a dataset using the Pandas `loc` and `iloc` accessors. The Iris dataset is used to demonstrate label-based selection, integer-position-based selection, conditional filtering, and column selection.

The project develops an understanding of DataFrame indexing and how to access relevant portions of a dataset for data analysis and machine learning.

2. Introduction

Data analysis often requires selecting only relevant rows and columns from a larger dataset. Pandas provides two useful accessors for this purpose: `loc` and `iloc`.

The `loc` accessor selects data primarily using labels, while `iloc` selects data using integer positions. Understanding these differences helps analysts retrieve the intended information accurately.

This project applies both accessors to the Iris dataset in Google Colab.

3. Objectives

- Understand label-based selection using `loc`.
- Understand position-based selection using `iloc`.
- Select specific rows and columns from a DataFrame.
- Select single and multiple columns.
- Filter rows based on a condition.
- Compare the behavior of `loc` and `iloc`.

4. Tools and Technologies

Python: Used to execute the data-selection operations.

Pandas: Used to store data in a DataFrame and select rows and columns.

NumPy: Supports numerical data handling.

Scikit-learn: Used to load the Iris sample dataset.

Google Colab: Provides the Jupyter Notebook environment for executing the code.

5. Dataset Description

The project uses the Iris dataset, which contains 150 observations.

The DataFrame contains five columns:

1. Sepal length
2. Sepal width
3. Petal length
4. Petal width
5. Species

The first four columns are numerical measurements, while the species column contains categorical labels.

6. Methodology

The project follows these steps:

1. Load the Iris dataset.
2. Convert it into a Pandas DataFrame.
3. Display the initial dataset records.
4. Select rows and columns using `loc`.
5. Select rows and columns using `iloc`.
6. Select individual and multiple columns.
7. Filter records using a Boolean condition.
8. Compare label-based and position-based indexing.
9. Display the selected records and dataset dimensions.

7. Label-Based Selection Using loc

The `loc` accessor uses row labels and column names to retrieve data.

For example:

`df.loc[0:4, ["sepal length (cm)", "petal length (cm)", "species"]]`

This selects rows with labels 0 through 4 and the three specified columns.

The ending label is included in a `loc` slice.

The project also uses `loc` to select particular row labels and to filter observations where the species equals setosa.

8. Position-Based Selection Using iloc

The `iloc` accessor selects records using integer positions.

For example:

`df.iloc[0:5, [0, 2, 4]]`

This selects the first five rows and columns at positions 0, 2, and 4.

Unlike `loc`, the ending position in an `iloc` slice is excluded.

9. Comparison Between loc and iloc

Feature	loc	iloc
Selection method	Labels	Integer positions
Row selection	Row labels	Row positions
Column selection	Column names or labels	Column positions
Slice endpoint	Included	Excluded
Conditional filtering	Supported	Usually requires positions or a Boolean array
Example	`df.loc[0:4, ["species"]]`	`df.iloc[0:5, [4]]`

Both accessors can select individual rows, multiple rows, single columns, multiple columns, or combinations of rows and columns.

10. Results and Observations

The dataset contains 150 rows and five columns.

The project demonstrates that:

1. `loc` selects records using labels and column names.
2. `iloc` selects records using row and column positions.
3. `loc` can select observations that satisfy a condition, such as a specific species.
4. Multiple columns can be selected by providing a list of column names or integer positions.
5. Label-based slices include their ending label, while integer-position slices exclude their ending position.

Insert actual notebook output screenshots to demonstrate these operations.

11. Applications

Row and column selection is useful for data cleaning, exploratory data analysis, feature selection, filtering records, preparing training data, and selecting variables for machine-learning models.

For example, an analyst can select only the measurement columns needed for analysis or filter records belonging to a particular category.

12. Limitations

This project focuses on basic row and column selection. It does not build a machine-learning model or perform advanced data cleaning and feature engineering.

13. Future Enhancements

The project could be extended to work with larger datasets, select features dynamically, filter multiple conditions, compare categories, and prepare selected columns for model training.

14. Conclusion

The project demonstrates the use of Pandas `loc` and `iloc` to select specific rows and columns. It highlights the difference between label-based and integer-position-based selection and shows how both methods support structured data analysis.

15. References

1. Pandas indexing documentation: https://pandas.pydata.org/docs/user_guide/indexing.html
2. Pandas documentation: https://pandas.pydata.org/docs/
3. Scikit-learn datasets: https://scikit-learn.org/stable/datasets/toy_dataset.html

17. Project Links

GitHub Repository: https://github.com/venky1318/Select_Dataset_Columns